

Use numpy to mask an image with a pattern?

Asked 13 years, 8 months ago Modified 4 years, 4 months ago Viewed 50k times



26



I'm using numpy to build pixel arrays. An 800x600 image is an 3-dimensional array of uint8, 800x600x3. I also have a similar array with a fixed pattern (a checkerboard, see [here](#)). I have another array, 800x600 of mask values. Where the mask is zero, I want to copy the pattern pixel to the image pixel. Where the mask is not zero, I want to leave the image pixel alone.

```
>>> image.shape
(800, 600, 3)
>>> chex.shape
(800, 600, 3)
>>> mask.shape
(800, 600)
```

This feels like it should work:

```
image[mask == 0, ...] = chex
```

but gives "ValueError: array is not broadcastable to correct shape".

What do I use to copy chex pixels to image pixels where mask is zero?

python image-processing numpy [Edit tags](#)

Share Edit Follow Close Flag

edited May 23, 2017 at 12:25



Community Bot

1 1

asked Jan 30, 2010 at 21:57



Ned Batchelder

365k 75 564 662

6 Answers

Sorted by: [Reset to default](#) Date modified (newest first)



4

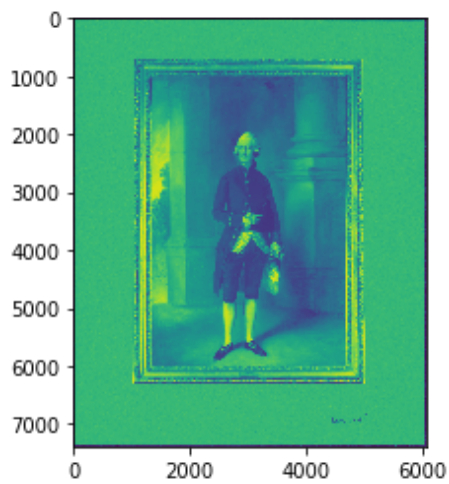


I found it easiest to create a mask wherein 1 = "pixel to keep" and 0 = "pixel to remove".

Then I multiplied my image by that mask to remove the unwanted pixels. Example to retain only the frame (outside) of a portrait:

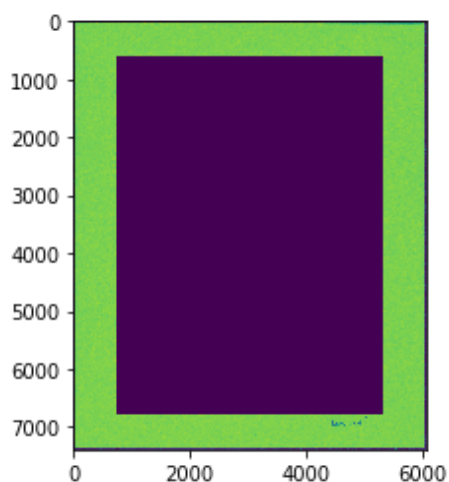
```
from scipy.misc import imread
import matplotlib.pyplot as plt
import numpy as np

im = imread('portrait.jpg', mode='L') # read in image
plt.imshow(im) # show the original image
```



```
mask = np.ones(im.shape) # create a mask with the image's shape
bw = 0.1 # identify border width and height as fraction of image size
bx = int(im.shape[1] * bw) # get the x dimension border width
by = int(im.shape[0] * bw) # get the y dimension border height
mask[bx:-bx,by:-by] = 0 # create a mask with 1 for border and 0 for inside

masked = im * mask # multiply `im` by the mask to zero out non-border pixels
plt.imshow(masked) # show the result of the masking operation
```



Share Edit Follow Flag

answered May 21, 2019 at 0:34



duhaime

25.8k

17

176

226



5

I wanted to illustrate an example using @unutbu answer. In this scenario, I have an image of a cat that I rotate. This rotation causes some black edges that look ugly especially when pasted on a non-black background.



```
import matplotlib.pyplot as plt
from scipy.ndimage import rotate
```



```
cat = plt.imread('cat.jpeg')
bg = plt.imread('background.jpeg')
```

```
rotcat = rotate(cat, angle=8, reshape=True) ## rotating creates some black edges
```

```
height, width, _ = rotcat.shape

bgcopy = bg.copy() ## create a copy of the background; paste on copy

x, y = 40, 50
bgcopy[x:x+height, y:y+width] = rotcat
plt.imshow('cat-on-bg-mask.jpg', bgcopy)
```



So, I find the areas of the mask and replace those values with the original background values

```
mask_ind = (bgcopy == 0)
bgcopy[mask_ind] = bg[mask_ind]
plt.imshow('cat-on-bg.jpg', bgcopy)
```



I should also note that `PIL.Image` (from the `Pillow` library) has the ability to paste an image onto another image with fewer steps.

Share Edit Follow Flag

answered May 21, 2018 at 1:16



Jon

2,403

1

26

34



I used arrays of 8x6x3, 8x6x3, and 8x6 to represent your image array, checker array, and mask array respectively.

0



first create mini-versions of your arrays:

```
mask = NP.random.random_integers(0, 1, 48).reshape(8, 6)
img = NP.random.random_integers(3, 9, 8*6*3).reshape(8, 6, 3)
chk = NP.ones((8, 6, 3))
```



all the work done in these two lines

```
mask = mask[:, :, NP.newaxis]
res = NP.where(mask==0, chk, img)
```

Share Edit Follow Flag

edited Jan 31, 2010 at 19:18

answered Jan 31, 2010 at 2:48



doug

69.2k

24

165

199



The shapes of the three images, mask.shape (8, 6), chk.shape (8, 6, 3) and img.shape (8, 6, 3) seems not to match the requirement of np.where. – [Gathide](#) May 20, 2017 at 5:05



```
idx=(mask==0)
image[idx]=chex[idx]
```

34



Note that image has shape (800,600,3), while idx has shape (800,600). [The rules](#) for indexing state



if the selection tuple is smaller than n, then as many : objects as needed are added to the **end** of the selection tuple so that the modified selection tuple has length N.



Thus indexing arrays have a sort of broadcasting ability of their own. idx 's shape gets promoted to (800,600,;)

Share Edit Follow Flag

edited Jan 31, 2010 at 12:45

answered Jan 31, 2010 at 0:17



unutbu

846k

184

1790

1681



Try:

2

```
image[mask[:] == 0,...] = chex[mask[:] == 0,...]
```



Share Edit Follow Flag

answered Jan 30, 2010 at 22:21



ezod
7,281 2 24 34



0



 **This answer is hidden.** This answer was [deleted](#) via review 7 years ago by [Vasfed](#), [Sharp Edge](#), [Whirlwind](#), [pppery](#), user3956566, [MSeifert](#).

This is not an answer to your question as posed, but in the [Python Image Library](#) (PIL), I think the function Image.composite does what you want.

Share Edit Follow **Undelete** Flag

answered Jan 30, 2010 at 23:33



Jive Dadson
16.8k 9 52 65

Comments disabled on deleted / locked posts / reviews